Notes-Gen-AI

System Requirements & Architecture Specification

1. Executive Summary

Notes-Gen-AI is an AI-powered platform engineered for real-time lecture recording, live speech transcription, automated structured note generation, note management, and interactive learning practice. The system utilizes a modern microservices ecosystem to process streaming audio and generate tailored educational content.

2. Core Product Flow

The primary user journey follows a sequential pipeline from media capture to interactive learning:

1. **Start Recording:** User initiates capture; UI requests microphone and display permissions.
2. **Media Capture & Chunking:** Browser captures audio/video via `MediaRecorder` and `Web Audio API`, sending small chunks continuously to the backend.
3. **Live Transcription:** Real-time processing via Faster-Whisper, streaming text back to the UI over WebSockets.
4. **Stop & Template Selection:** Recording halts; user selects a formatting template (e.g., Cornell notes, summary).
5. **AI Generation:** Gemini AI compiles structured sections, key concepts, questions, and flashcards.
6. **Save & Organize:** Notes are persisted in custom folders within "My Notes" for future review or export.

3. Architectural Principles

The system is built on scalable, distributed architecture principles:

- **Domain-Driven Design:** Each microservice owns its specific business domain and isolated database schema.
- **Event-Driven Orchestration:** RabbitMQ handles long-running workflows (e.g., `TranscriptionCompleted`, `NoteGenerationCompleted`).
- **Stack Alignment:** AI processing requiring C++ bindings operates in Python, while transactional and business logic operates in Java/Spring Boot.
- **Stateless Edge:** The React frontend communicates strictly through a Spring Cloud Gateway, which manages CORS, security, and routing.

> **Architectural Update:** Per specific project requirements, the **Notes Generation Service** will be implemented in Java (Spring Boot) leveraging Spring AI, transitioning away from the default Python deployment for this specific module to align with team expertise.

4. Microservices Matrix

The backend is divided into specialized modules. Service communication relies on REST for immediate operations and RabbitMQ for asynchronous tasks.

Service Name	Stack	Core Responsibilities
API Gateway (Port: 8080)	Java Spring Cloud	Single entry point, routing, rate limiting, security headers, CORS, WebSocket proxying.
User Service (Port: 8081)	Java Spring Boot	Registration, JWT/OAuth2 login, RBAC, profiles, preferences. Backed by PostgreSQL.
Transcription Service (Port: 8082)	Python FastAPI	Receives A/V chunks, extracts audio via FFmpeg, transcribes via Faster-Whisper, broadcasts WebSocket updates.
Notes Generation (Port: 8083)	Java Spring Boot	Ingests transcripts, builds Gemini prompts via Spring AI, generates structured notes and flashcards.
Templates Service (Port: 8084)	Java Spring Boot	CRUD for system/user prompt definitions, stores template metadata.
Notes Management (Port: 8085)	Java Spring Boot	Folder structures, versioning, search, archiving, document exports (PDF, Markdown, DOCX).
Practice Service (Port: 8086)	Java Spring Boot	Generates and scores MCQs, True/False, and flashcards from saved notes. Tracks progress.
Payment Service (Port: 8087)	Java Spring Boot	Order creation, UPI/Razorpay integration, server-side webhook validation.
Credit Management (Port: 8088)	Java Spring Boot	Atomic credit reservation via Redis, deduction on success, release on failure.
Frontend UI (Port: 3000)	React Vite	Zustand state management, MediaRecorder API, Web Audio API mixing, responsive UI.

5. Infrastructure & Storage Strategy

Data ownership strictly follows the database-per-service pattern. Shared infrastructure is utilized for caching, messaging, and large blob storage to ensure database performance remains optimal.

Infrastructure Component	Port	Usage in System
PostgreSQL	5432	Primary relational store for Users, Templates, Note Metadata, Payments, and Credits.
Redis	6379	Distributed caching, JWT token blacklisting, and atomic locks for credit reservation.
RabbitMQ	5672	Message broker routing events like `TranscriptionCompleted` and `PaymentCompleted`.
MinIO / AWS S3	9000	Object storage for raw audio/video chunks and exported PDF/DOCX generated files.
MongoDB (Optional)	27017	Secondary document store if note bodies become too large for PostgreSQL text fields.

6. Development Roadmap

- **Phase 1 (Core):** User Service, API Gateway, React UI, Transcription (Python), Notes Generation (Java), Notes Management.
- **Phase 2 (Organization):** Templates Service, Folders, Search, Export/download logic.
- **Phase 3 (Learning):** Practice Service (Quizzes, Flashcards, Progress tracking).
- **Phase 4 (Monetization):** Credit Management, Payment Service integration (UPI).
- **Phase 5 (Production):** Docker orchestration, Prometheus/Grafana observability, CI/CD pipelines.